

CIMAS: A Multilingual Semantic-Driven Model for Code Idiom Mining, Abstraction and Synthesis

Wenxin Yao

The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China
wenxinyao@zju.edu.cn

Abstract—Code idioms are widely recognized as common, reusable templates, patterns, or best practices in software development, which significantly enhance programming efficiency and code quality. In this work, we propose the CIMAS model, which adopts a modular framework to perform idiom mining in a structured and progressive manner. Specifically, CIMAS comprises four sub-modules: idiom representation identification, large-scale idiom code mining, idiom elements abstraction, and idiom judgment and synthesis with large language models. To validate the effectiveness of CIMAS, we apply the mined idiom repository to two downstream tasks: unit test generation and programming specification recognition. Experimental results demonstrate that code idioms effectively enhance semantics. The number of AST nodes increases by an average of 38.8% and 28.4% for the two downstream tasks, respectively.

I. INTRODUCTION

Code idioms are commonly defined as prevalent programming patterns or best practices that encapsulate clear and semantically rich intentions [1], [2]. In recent years, code idioms have attracted attention from both academia and industry due to their significance in software engineering. They not only enable more concise and efficient problem-solving but also enhance code readability and decrease semantic ambiguity. This aligns with agile development principles [3], enhancing both productivity and team cohesion.

We propose an innovative multilingual Code Idiom Mining, Abstraction, and Synthesis (CIMAS) model, which automatically extracts previously undefined code idioms from large-scale code repositories. Thanks to its modular design, CIMAS can be easily adapted to different programming languages by making minor adjustments to the first sub-module. Results demonstrate that the mined code idioms exhibit high quality and are applicable to a variety of downstream tasks.

II. BACKGROUND AND RELATED WORK

Research on code idioms generally follows two main directions. The first defines idiom types in advance and applies rule-based pattern matching grounded in the syntactic features of specific programming languages. For instance, Holeman et al. [4] constructed a catalog of common idioms to analyze the structure of generic libraries. Wikipedia also lists 75 types of coding idioms, many of which are tailored to C++, such as expression templates and smart pointers [5]. In the Python ecosystem, *RIdiom* refactors non-idiomatic code into idiomatic patterns, focusing on nine syntax-driven idioms [2], [6]. With the advent of LLMs, such refactoring can now be scaled more efficiently [7].

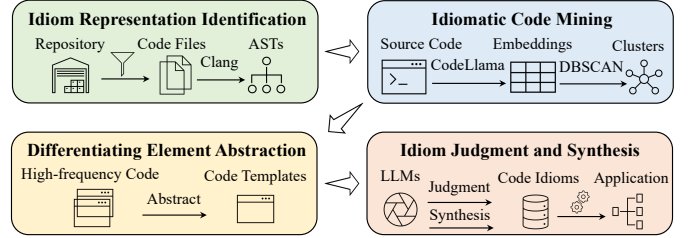


Fig. 1. The overall framework of the CIMAS model

An alternative research paradigm focuses on code semantics rather than predefined idiom types, aiming to discover idioms through statistical or learning-based models. *Haggis* [8] initiated this direction by mining recurring structural patterns using non-parametric Bayesian probabilistic tree substitution grammars. Building on this, *IdioMine* [1] effectively extracts cross-line and interleaved idioms by leveraging data-driven control chains from program dependency graphs.

III. APPROACH: CIMAS

The overall framework of CIMAS is shown in Fig. 1.

A. Idiom Representation Identification

This sub-module focuses on dataset construction and pre-processing, mainly using static analysis to locate and extract potential code idiom candidates. We conduct experiments on four popular and well-maintained C++ open source repositories on GitHub. Subsequently, we employ Clang as the static analysis tool to transform source code into ASTs.

To capture code idioms across different levels, we categorize code segments into the following three granularities based on AST node types. (1) The *Function-Level* targets entire functional modules and represents the largest granularity. (2) The *Statement-Level* is the most fine-grained, focusing on routine coding behaviors. (3) The *Range-Level* captures syntactically continuous blocks such as control structures, loop bodies, and exception handlers.

B. Idiomatic Code Mining

This sub-module identifies and extracts idiomatic code segments. (1) We employ embedding techniques to capture the semantic content of code segments. We use the CodeLlama-7B model [9], which has shown strong performance in tasks such as code completion, semantic search, and retrieval. (2) Subsequently, we adopt a density-based clustering

TABLE I
THE PERFORMANCE OF CIMAS

Repo	IC	ISP	F1	ANI ₁	ANI ₂
TrafficMonitor	0.50	0.68	0.58	39.1%	24.5%
qBittorrent	0.51	0.35	0.42	32.3%	28.1%
envoy	0.52	0.73	0.61	41.8%	33.6%
react-native	0.55	0.70	0.62	42.1%	27.2%
Average	0.52	0.62	0.56	38.8%	28.4%

approach, DBSCAN, to group similar code segments within each repository. After clustering, code segments located at cluster centers are selected as representative idiomatic patterns.

C. Differentiating Element Abstraction

This sub-module identifies potential code templates within the mined idiomatic segments. Building on the clustering results from the previous module, it detects varying elements across similar code instances, such as constants, variable names, or control expressions. These elements are then abstracted to generate generalized idioms that retain essential structural characteristics while discarding contextual variations, thus promoting broader reusability.

In this sub-module, we abstract idiomatic code segments identified from the previous stage. For each high-cohesion cluster, we extract the constituent code segments and treat them as strong candidates for idiomatic templates. Each segment is then tokenized, and tokens are either retained or abstracted based on their consistency with the cluster center. Using a Sliding Window of size 2, we compute token sequence consistency across the cluster. Tokens with a match ratio above 0.8 are preserved as common; others are abstracted. Finally, we define five abstract types, namely $\langle \text{STR} \rangle$, $\langle \text{CONST} \rangle$, $\langle \text{TYPE} \rangle$, $\langle \text{VAR} \rangle$ and $\langle \text{FUN} \rangle$.

D. Idiom Judgment and Synthesis

This sub-module leverages the capabilities of LLMs to finalize the identification of code idioms. We use ChatGPT-4o to conduct the following two-step experiment.

(1) Idiom Judgment: The first step evaluates whether a candidate code segment exhibits meaningful and clear semantics. The motivation is that idiomaticity and semantic clarity are two essential properties of code idioms. By prompting ChatGPT-4o to analyze the expressiveness and intent of the code segment, we can determine whether it qualifies as a proper code idiom.

(2) Idiom Synthesis: The second step aimed at improving idiom quality by combining semantically related code segments as a block. First, only code segments from the same AST are considered, ensuring they originate from the same source region and share contextual relevance. Then we employ ChatGPT-4o to evaluate whether a meaningful semantic association exists between them. The semantically associated segments are synthesized into a more comprehensive idiom.

IV. EVALUATION AND APPLICATION

The effectiveness of the proposed model is demonstrated through representative examples of mined code idioms. As

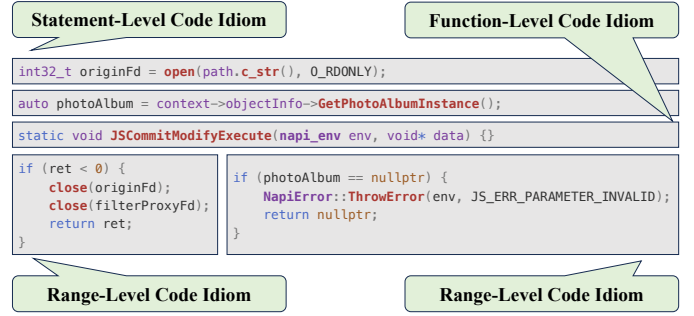


Fig. 2. **Idiom Judgment** results: code idioms of different granularity

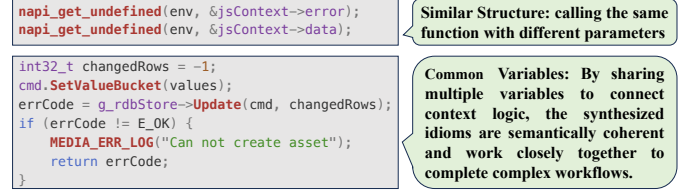


Fig. 3. **Idiom Synthesis** results: combinations of related code idioms

illustrated in Fig. 2, the model is capable of extracting code idioms across three distinct levels of granularity. Furthermore, Fig. 3 shows how related idioms can be synthesized into semantically coherent and frequently used idiomatic patterns. This synthesis leverages principles such as shared variables, structural similarity, and common co-occurrence.

Code idioms facilitate software understanding and maintenance, serving as a structured knowledge base for diverse downstream tasks. We demonstrate the advantages of code idioms in two scenarios: **Unit Test Generation** and **Programming Specification Recognition**.

Following prior evaluation protocols [1], [8], we adopt three primary metrics: Idiom Coverage (IC), defined as the proportion of AST nodes in a function matched by mined idioms; Idiom Set Precision (ISP), denoting the percentage of training idioms reappearing in the test set; and their harmonic mean (F1), which captures the trade-off between IC and ISP. Additionally, we use the number of AST nodes to quantify semantic richness, with ANI_1 and ANI_2 representing AST Node Improvements on task 1 and task 2, respectively. The results in Table I show that the mined code idioms are of high quality based on semantic and high-frequency features, and effectively enhance semantics on different downstream tasks.

V. CONCLUSION

This paper proposes CIMAS, a multilingual modular framework for mining code idioms from repositories. CIMAS employs a sequential pipeline that identifies, generalizes, and evaluates idiomatic patterns through static analysis, embedding techniques, and large language models. The resulting idiom library supports downstream tasks, such as unit test generation and programming specification recognition, achieving average AST node increases of 38.8% and 28.4%, respectively.

REFERENCES

- [1] Y. Yang, X. Hu, X. Xia, D. Lo, and X. Yang, “Streamlining java programming: Uncovering well-formed idioms with idiomine,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.
- [2] Z. Zhang, Z. Xing, X. Xia, X. Xu, and L. Zhu, “Making python code idiomatic by automatic refactoring non-idiomatic python code with pythonic idioms,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 696–708.
- [3] S. Al-Saqqah, S. Sawalha, and H. AbdelNabi, “Agile software development: Methodologies and trends,” *International Journal of Interactive Mobile Technologies*, vol. 14, no. 11, 2020.
- [4] R. N. Holeman, “Identifying programming idioms in c++ generic libraries,” Master’s thesis, Kent State University, 2009.
- [5] Wikibooks contributors, “More c++ idioms,” 2024, https://en.wikibooks.org/wiki/More_C%2B%2B_Idioms.
- [6] Z. Zhang, Z. Xing, X. Xu, and L. Zhu, “Ridiom: Automatically refactoring non-idiomatic python code with pythonic idioms,” in *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 2023, pp. 102–106.
- [7] Z. Zhang, Z. Xing, X. Ren, Q. Lu, and X. Xu, “Refactoring to pythonic idioms: A hybrid knowledge-driven approach leveraging large language models,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1107–1128, 2024.
- [8] M. Allamanis and C. Sutton, “Mining idioms from source code,” in *Proceedings of the 22nd acm sigsoft international symposium on foundations of software engineering*, 2014, pp. 472–483.
- [9] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.